1 Project Overview

The design project of this semester's Embedded Systems Lab makes extensions to your previously developed hardware and software. Your task is to develop a client in a distributed system. The main function of the system is to synchronize a data set between all clients. Each client can modify the data set using the switches and buttons introduced in Warmup 2. To indicate the state of each client, the data is to be displayed using the scrolling text display from Warmup 4. The individual data can be manipulated using the buttons and switches as inputs. For synchronization, a simple CAN-based protocol is provided to exchange data. Fig. 1 shows the general structure of the system. The project specifications are described in more detail below.

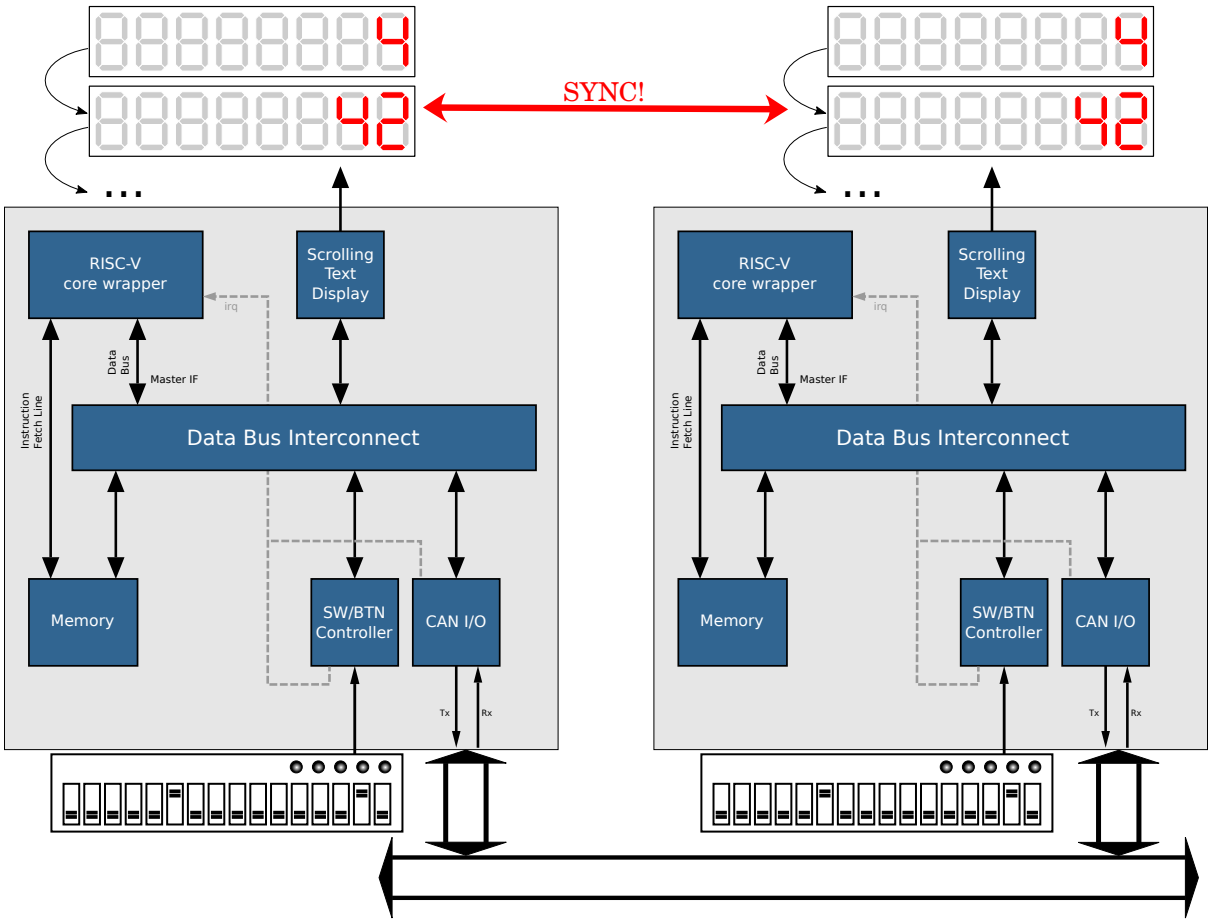

Figure 1

Before you start with the project, make sure you have the latest version of the SoC by running:

```
$ git pull origin master
```

2 The 'Project' Aspect

This is a development project. Plan the architecture of your system in advance. Identify tasks and distribute them among group members. Create a schedule for the tasks, with the allotted time for each task and for integration, testing and debugging. Many details about the implementation are left out on purpose. It is up to you to find suitable solutions for your individual design.

You will present your plan during a review meeting. Refer to the "general information" slides in the OLAT download section for guidelines on the presentation. (You will need to make an appointment for the review meeting.)

3 Development Target

As mentioned before, this project is about developing a client in a distributed system. Each group has to develop and implement such a client. Each client has a data set consisting of 16 individual 5 bit values that encode hexadecimal characters, that are displayed on the scrolling text display. The data is shared by all clients in the system. Clients can make changes to the data set using their buttons and switches. When a client changes the data, it must synchronize the change with all other clients in the system. This is to be done using the CAN bus and the CAN bus controller that must be integrated into the system. Note that only the scrolling speed and the data set must be synchronous - not the exact time points of the changes on the display. Details on the CAN controller and setup are given in section 5. The specific protocol for distributing updates is explained in depth in section 6.

4 Switches and Buttons as System Input

The switches component developed in Warmup 2 is used as an input device. The peripheral component manages a set of 5 buttons and 16 switches. In this project, the switches are interpreted as data items and the buttons are used to implement the interface for the following three control functions:

1. Adding an 5 bit data item (bottom 5 switches) to the data set
2. Clearing the data set
3. Updating the scrolling speed (top 16 switches)

Each control function has to be assigned a button. Choosing which control function is assigned to which button is up to you. The data items read from the switches are used for two purposes. They are interpreted as the hexadecimal characters added to the data set (bottom 5 switches) or determine the scrolling speed of the scrolling text display (top 16 switches). For the scrolling speed, the 16 bit value obtained from the switches is interpreted as the 16 most significant bits of a 32 bit unsigned binary number. The remaining 16 least significant bits are filled with zeros. This ensures that the scrolling speed is in a human observable range, similar to the counter value in Warmup 1. This value should be adjusted for simulation.

5 The CAN Controller

The lab's git repository contains the hardware design for a CAN controller. The controller is functionally compatible with the *SJA1000* (and *PCA82C200*). Use only the basic transmit functions and do not use the PeliCAN mode.
The controller is written in Verilog, and has an 8-bit Wishbone interface (byte-accessible only) and several memory-mapped control and status registers. A hardware wrapper in SystemVerilog is provided [1] to connect the device to the rest of the system. The wrapper does not extend its data interface beyond 8 bits, so only byte-sized accesses to the device are allowed. The wrapped CAN controller must be integrated into the system, following the same steps as in previous assignments.

[1] *<project root>/soc/peripherals/CAN/can_wrapper.sv*

5.1 The CAN-Transceiver Pmod

To physically interact with the CAN network, an electrical adapter is needed. This adapter chip implements the physical layer of the CAN protocol. It consists of an RS485 driver on a separate circuit board that has a convenient connector adapter for the board. These are called “Pmod” adapter- or I/O-boards.
The converter-circuit-board can be connected to any of the Pmod connectors on the FPGA board. Add appropriate I/O-signals to your top module of the SoC. Depending on which connector you choose, you have to map connector data pins to port signals in your top module. For this purpose, edit your `.xdc` file by adding the chosen connectors pins and mapping them to signals. As noted in the data sheet, the maximum bit rate is 16 Mbit per second. Due to prior experience, it is recommended to choose a value significantly lower than the maximum.

6 The Data Protocol

The protocol for distributing updates has three different types of frames. Each frame type signals one of the following update functions: adding characters to the data set, clearing the data set and setting the scrolling speed of the display.
Each time a client makes a change to the data set, it needs to notify the other clients in the system.
To achieve this, each change should trigger the sending of a CAN frame, while simultaneously listening the CAN bus for updates.
For adding a new character, *data add frames* are used. Their composition is explained in the following. The ID of an update frame consists of a 6 bit fixed update code `"000001"`, and a 5 bit number that is unique to the client in the system. The data field consists of 2 byte, a frame type byte, which is all zero for data add frames, and the updated 5 bit character value. This data frame is schematically illustrated in Figure 2.

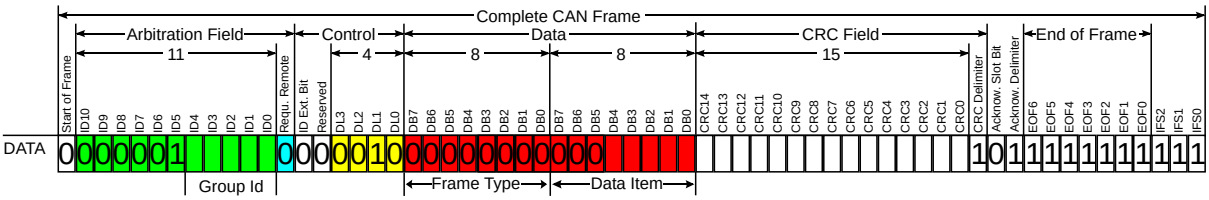

Figure 2: The “data add”-frame illustrated

Upon receiving an *data add frame*, the client should add the 5 bit character to its data set. If the buffer is already full, the first element should be overwritten. Note that this behavior is compliant with the functionality of the ring buffer in the scrolling text display component.
The next frame type, *data set clear*, is sent when a client receives a clear command from the buttons. As the name suggests, both, sender and receiver should make use of the clear buffer functionality display controller. This frame type only consists of one data byte (the frame type byte) set to `"00000001"`. Figure 3 shows the schematics of this frame type.

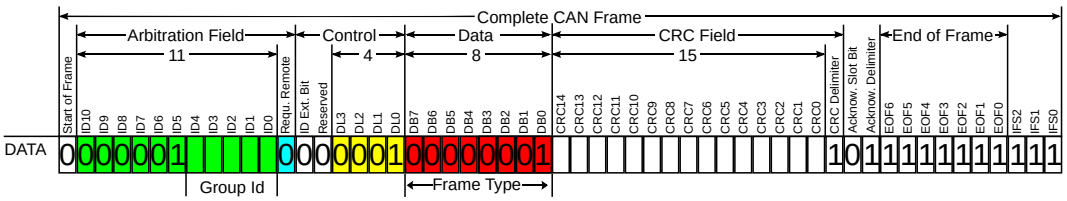

Figure 3: The “data set clear”-frame illustrated

The third frame type, illustrated in Figure 4, is used for updating the scrolling speed of the display. Like the *data add frame*, the *update frequency* uses 3 bytes, a frame type byte set to `"00000011"` and, in the following two bytes, bits 16 to 31 of the content of the 32 bit scrolling counter value register `cnt_value`.

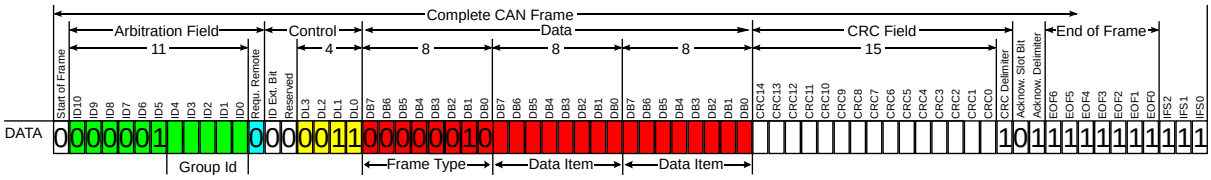

Figure 4: The “update frequency”-frame illustrated

Multiple clients initiating updates at (roughly) the same time can lead to race conditions. Think on how they can occour and how to avoid them.

7 Testing

For a successful verification it is necessary to simulate a system with multiple clients. For this purpose, several instances of the SoCs should be created, and their CAN connectors properly connected. As already mentioned above, the scrolling speed should be adjusted to decrease the size of the simulation window.

8 Initialization Problem

The system is asynchronous and clients may be added or removed from the system at any time. This leads to the problem that a new client does not have a synchronized copy of the data set.
Think about possible solutions to this problem. Come up with a possible extension of the previously established protocol, and present it in the review meeting (See Sec. 9.1). You do not need to implement a solution.

9 Completion

In the beginning of the Design Project, you are required to develop an overview of the task and present a project plan in the review meeting. To schedule a review meeting you should make an appointment by writing an email to Prof. Stoffel.
After the review meeting, you have to complete the project, followed by a demonstration to a teaching assistant. The demonstration will be divided into two parts: a demonstration of the simulation of your design and an FPGA implementation. The simulation should show at least two clients displaying their patterns in synch. During the simulation, each client should perform each of the update function described in Sec. 6 at least once. After the successful demonstration of the simulation, you have to present your solution on an FPGA board. Additional FPGA boards can be provided to be used as additional peers in the CAN network. The deadline for the project is the last week of the lecture periode.

9.1 Review Meeting

In the review meeting, you need to present your plan in a way that conveys that your understanding of the given task is sufficient, and present your plan on how to proceed. Among the mandatory items your project plan needs to incorporate are:

- Hardware-software-partitioning
- Identification/partitioning into individual tasks
- Time line for implementation and testing
- Distribution of the individual tasks among group members